

Now let's look at how a device like a USB thumb drive can be used in a cloud set-up.

Peripheral virtualisation

In the course of using a computer today, we connect devices like a thumb drive, printer, Web camera, etc, besides a mouse and keyboard, to a PC. But in a cloud computing set-up, when the computing service is in a remote location, how does an end user use such peripherals? In an ideal cloud set-up, these devices get



virtually
connected

to a cloud
server even
though the device

is connected to the
client, which is at the
user's home set-up. These
ideal cloud servers and home
set-ups are available in Linux (and
also other operating systems).

As a Linux hacker, you should be aware of what is happening inside the kernel driver layer that enables virtual peripheral connectivity. To understand this, let's look at how to modify the Linux kernel and virtually connect USB devices to a cloud server. One important aspect to take care of during virtualisation is that timing requirements have to be satisfied as per the protocol specification, besides transparency to the user that it's a virtual device. Thus, when virtualising a USB device, care has to be taken to ensure the quality of service and that the user doesn't feel like the device is remotely connected.

To virtualise a USB device in a remote cloud server, 'information' that helps virtualise the device has to be made available over the IP network. The current driver design has to be modified to suit this requirement. To understand this, let us first analyse the USB driver architecture and then understand where to hook on this information to the IP layer.

A Linux USB host driver consists of three major layers. The first low-level module is the Host Controller Driver (HCD), which implements hardware specific routines. The next portion is the USB Core Driver implementing USB-specific functions. The final layer is the application layer. Figure 3 illustrates the USB driver architecture.

When our client desktop connects to a cloud server over the Internet, the device information and the data from the USB device has to be transferred through the network to a server on the remote cloud. This device information and data has to be tapped from the appropriate layer so that the virtual device can function properly. Figure 4 shows how the information could be tapped in a USB device.

USB virtualisation in your home lab

Having looked at all internal kernel changes, let us now explore how to set up a cloud-like environment in your home lab. To do the set-up, prepare two Linux machines and a USB peripheral like a thumb drive.

The open source ecosystem is rich with projects that allow us to explore multiple ideas and this applies to the current requirement too. The USB/IP project will help you to understand a possible approach of how a USB device can be virtualised across a network. The USB/IP ideally shares USB devices between computers by encapsulating USB transfers into TCP/IP payloads and transmits them between computers.

Here is a quick overview of how to set up a USB/IP at home. [For detailed steps, refer the README file or hack the source code.] In a USB/IP, the PC that exports a virtual device is referred to as a server and the PC that uses a virtual device, as the client.

The first step in the process is to download the source packages from <http://usbip.sourceforge.net> in both the computers. Compile the source in the driver folder and install the server drivers following the steps given in the README file [<http://usbip.svn.sourceforge.net/viewvc/>]

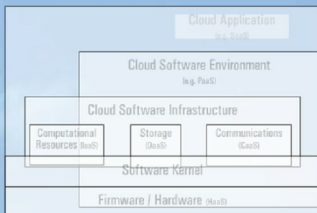


Figure 1: Five layers of cloud computing